

Multivariate Regression Analysis of LLM Hardware Performance

DS3005 — Computational Algorithms in Data Science

Lakshya Gupta · Krissh Modi · Sarthak Goel

Plaksha University

1. Introduction, Background, and Motivation

Large language models (LLMs) have proliferated rapidly across architectures ranging from early transformer-based designs (GPT-2, GPT-NeoX) to modern instruction-tuned families (LLaMA, Mistral). Deploying these models at scale requires careful understanding of their hardware footprint: how fast they run (throughput in tokens/s), how much GPU memory they consume (peak memory in MB), and how accurately they perform on standard benchmarks (score in %).

These three outcomes are not independent. A model that runs faster may use less memory, and a larger model generally scores higher but demands more resources. Practitioners selecting models for deployment must navigate this three-way trade-off.

Standard practice fits one regression per response. However, when multiple responses share predictors and are correlated, a joint multivariate model is statistically more principled: it captures cross-response structure through the residual covariance matrix Σ_{hat} , and a single coefficient matrix B explains all three outcomes simultaneously. This also enables direct diagnosis of which predictors explain shared versus response-specific variance.

Motivation:

- Apply multivariate multiple linear regression to a real-world LLM benchmarking dataset.
- Compare OLS, Ridge, and Elastic Net estimators and articulate when each is appropriate.
- Show how adding categorical architecture predictors changes model fit and the interpretation of size effects.
- Demonstrate that a non-diagonal residual covariance matrix justifies the joint modelling approach over three separate regressions.

2. Problem Statement

Given: A dataset of 370 LLM benchmark runs covering 12+ architecture families, with predictors including model size in billions of parameters (Params_B), numerical precision in bits (Bits), architecture family (Type), and inference backend (Backend).

Task: Simultaneously predict three hardware performance responses:

Response	Unit	Description
Throughput	tokens/second	Inference speed
Peak Memory	MB	Peak GPU memory during inference
Score	%	Aggregate benchmark accuracy

Research questions:

1. How much variance in each response is explained by model size and precision alone (base model)?
2. Does architecture family add significant explanatory power beyond parameter count (extended model)?
3. Which regularisation strategy — Ridge or Elastic Net — performs better when categorical predictors are included?
4. What do the off-diagonal entries of the residual covariance matrix reveal about the three responses, and does controlling for architecture change this structure?

3. Technical Details: Methodology, Formulas, and Assumptions

3.1 The Multivariate Regression Model

All three responses are modelled simultaneously:

$$Y = X B + E$$

where Y ($n \times 3$) is the response matrix ($n=370$, 3 outputs), X ($n \times p$) is the design matrix ($p=3$ base / $p=14$ extended, including intercept), B ($p \times 3$) is the coefficient matrix to be estimated, and E ($n \times 3$) is the residual error matrix.

The key benefit over three separate regressions is access to the **error covariance matrix**:

$$\text{Sigma_hat} = (E^T E) / (n - k - 1), \quad E = Y - Y_{\text{hat}}$$

whose off-diagonal entries quantify residual dependencies between the three outputs.

3.2 OLS Estimator (Gaussian Baseline)

The standard closed-form OLS solution:

$$B_{\text{hat}} = (X^T X)^{-1} X^T Y$$

When $X^T X$ is singular (multicollinear or rank-deficient), the Moore-Penrose pseudoinverse `np.linalg.pinv(X)` is used as a numerically stable fallback:

$$B_{\text{ols}} = \text{np.linalg.pinv}(X) @ Y$$

3.3 Ridge Regression

Ridge adds an L2 penalty to stabilise the inversion:

$$B_{\text{hat_Ridge}} = (X^T X + \lambda I)^{-1} X^T Y, \quad \lambda = 0.1$$

This shifts all eigenvalues of $X^T X$ upward by λ , guaranteeing invertibility. The intercept is excluded from the penalty (`I_ridge[0, 0] = 0`). Regularisation here is not primarily for variable selection — it ensures numerically stable coefficient estimates when low-frequency categorical predictors are included:

$$\begin{aligned} I_{\text{ridge}} &= \text{np.eye}(X.\text{shape}[1]); I_{\text{ridge}}[0, 0] = 0 \\ B_{\text{ridge}} &= \text{np.linalg.inv}(X.T @ X + 0.1 * I_{\text{ridge}}) @ X.T @ Y \end{aligned}$$

3.4 Elastic Net

Elastic Net combines L1 and L2 penalties, implemented via `MultiOutputRegressor(ElasticNet(alpha=1.0, l1_ratio=0.5, fit_intercept=True))` — one model per response. This is the key distinction from Ridge: Elastic Net fits each response independently and cannot capture inter-response correlations through a shared B . It is included for comparison.

3.5 Evaluation

- **In-sample R^2** : fraction of variance explained on training data.
- **5-fold cross-validated R^2** : generalisation estimate. Implemented with `kFold(n_splits=5, shuffle=True, random_state=42)`, computed per response and reported individually and as a mean.

Since the three outputs have very different scales, response standardisation is not required — OLS and Ridge are equivariant under response scaling. The residual covariance is interpreted through its normalised correlation form.

3.6 Assumptions

1. **Linearity**: conditional mean of each response is linear in the predictors.
2. **Gaussian errors**: each row of E follows a multivariate Gaussian with covariance Σ — sufficient for OLS/Ridge to be BLUE.
3. **No perfect multicollinearity**: managed by Ridge and `drop_first=True` in dummy encoding.
4. **Homoscedasticity**: acknowledged as a limitation for Throughput (heteroscedastic residuals visible in Figure 3).

4. Data Description and Source

Source: Open LLM-Perf Leaderboard — a public Hugging Face dataset recording inference performance of open-source LLMs on standardised hardware.

URL: <https://huggingface.co/datasets/optimum/llm-perf-leaderboard>

After parsing and dropping rows with missing values, **370 valid observations** are retained.

Column	Processing	Final Feature
Class	<code>clean_params()</code> : "7B" to 7.0, "350M" to 0.35	Params_B (continuous)
Dtype	<code>clean_dtype()</code> : float16→16, float32→32, int8→8, int4→4	Bits (continuous)
Type	<code>get_dummies()</code> , <code>drop_first=True</code> (ref: Baichuan)	Binary dummies
Backend	<code>get_dummies()</code> , <code>drop_first=True</code> (ref: onnxruntime)	1 binary dummy
Throughput (tokens/s)	No transformation	Output y1
Peak Memory (MB)	No transformation	Output y2
Score (%)	No transformation	Output y3

Unique Bits values in dataset: {4, 8, 16, 32} — `clean_dtype()` handles the full quantisation spectrum.

Architecture families present: Baichuan (reference/dropped), ChatGLM, CodeGen, Falcon, GPT-2, GPT-BigCode, GPT-J, GPT-Neo, GPT-NeoX, LLaMA, MPT, OPT, XGLM, plus Backend (pytorch vs onnxruntime).

5. Results of the Analysis

5.1 Base Model: 2 Predictors (Params_B, Bits)

5.1.1 Coefficient Matrix — OLS (Gaussian Baseline)

From notebook Step 4 (`B_ols = np.linalg.pinv(X) @ Y`):

	<i>Throughput (TPS)</i>	<i>Peak Memory (MB)</i>	<i>Score (%)</i>
Intercept	53.96	-2260.68	36.17
Params_B	-2.41	+943.07	+0.58
Precision (Bits)	+1.71	+366.35	-0.04

(Ridge coefficients at $\lambda=0.1$ are numerically identical at this predictor count — confirmed in notebook Step 5.)

Interpretation:

- Every additional 1B parameters reduces throughput by ~2.41 tokens/s, adds ~943 MB of memory (~1 GB/B, consistent with float16 weight storage), and improves benchmark accuracy by ~0.58%.
- Higher precision (32-bit vs 16-bit) adds ~366 MB expected memory overhead per bit-width unit with negligible accuracy effect (-0.04% per bit).

5.1.2 In-Sample and 5-Fold CV R^2

From notebook Steps 7 and 9 (`kFold(n_splits=5, shuffle=True, random_state=42)`):

<i>Output</i>	<i>In-sample R^2</i>	<i>CV R^2</i>
Throughput (TPS)	0.138	0.1193
Peak Memory (MB)	0.550	0.5216
Score (%)	0.366	0.3674

All three estimators (OLS, Ridge, Elastic Net) produce effectively identical CV R^2 in the base model — OLS/Ridge: 0.1193/0.5216/0.3674; Elastic Net: 0.1195/0.5222/0.3705. The small gap between in-sample and CV R^2 confirms no overfitting with only 2 predictors.

- **Throughput CV $R^2 = 0.12$** is weak — inference speed is heavily influenced by GPU batch size, kernel optimisation, and hardware generation, none captured here.
- **Memory CV $R^2 = 0.52$** is moderate — parameter count is a strong driver of memory footprint.
- **Score CV $R^2 = 0.37$** is moderate — consistent with LLM scaling laws (Hoffmann et al., Chinchilla), where model size is a primary but not exclusive predictor.

5.1.3 Residual Error Covariance Matrix

From notebook Step 8 (`Sigma_hat = (E_hat.T @ E_hat) / (n_samples - k - 1)`):

Residual **correlation** matrix (normalised `Sigma_hat`):

	<i>Throughput (TPS)</i>	<i>Peak Memory (MB)</i>	<i>Score (%)</i>
Throughput	1.0000	-0.6315	-0.4174
Memory	-0.6315	1.0000	+0.3290
Score	-0.4174	+0.3290	1.0000

The strong off-diagonal values confirm the three responses are **not independent** after conditioning on Params_B and Bits — directly justifying the multivariate modelling approach:

- **Throughput $\leftrightarrow$ Memory ($\rho = -0.63$):** Models faster than predicted tend to use less memory. Architecture family is the hidden driver of this coupling.
- **Throughput $\leftrightarrow$ Score ($\rho = -0.42$):** Faster-than-predicted models score lower — speed comes at an accuracy cost.
- **Memory $\leftrightarrow$ Score ($\rho = +0.33$):** More memory than predicted correlates with higher score — larger models are more capable.

5.1.4 Residual Plots (from Notebook)

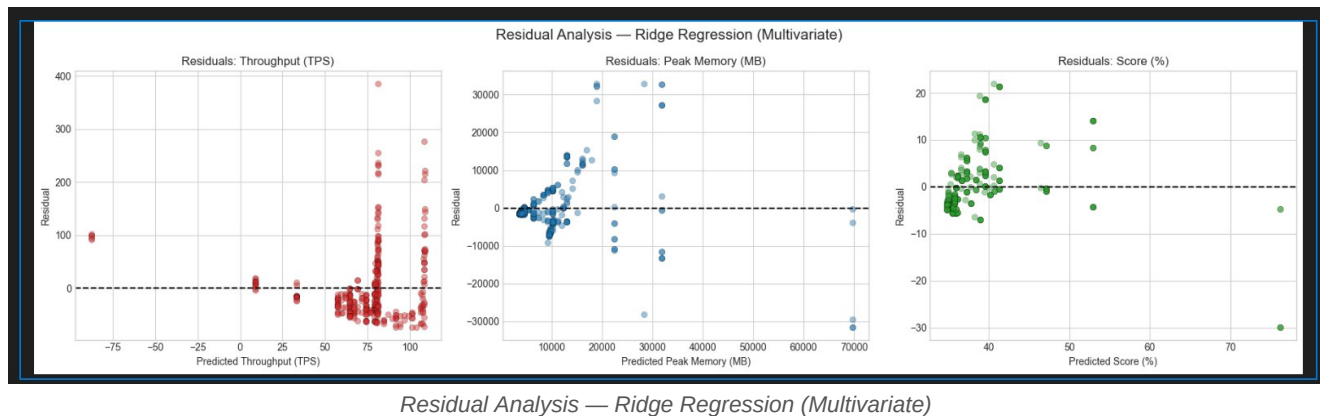


Figure 1. Residual plots for the Ridge base model (from notebook Step 9). Key observations:

- **Throughput residuals (left):** Clear heteroscedasticity — residual variance fans out at higher predicted values (~75–120 TPS). A log-linear model would address this.
- **Memory residuals (centre):** Distributed around zero but with large-magnitude outliers at very high predicted values (>60,000 MB) — extreme large-model runs are poorly captured.
- **Score residuals (right):** Well-behaved — roughly symmetric around zero across the full prediction range (~35–75%).

5.2 Extended Model: Architecture Type as Categorical Predictors

Motivation: The 2-predictor model ignores architecture family. GPT-2, LLaMA, GPT-NeoX, and OPT have fundamentally different attention mechanisms, vocabulary sizes, and positional encoding schemes, all affecting throughput, memory, and accuracy independently of parameter count.

Encoding (notebook Step 10):

```
type_dummies = pd.get_dummies(df_ext['Type'], prefix='Type', drop_first=True).astype(float)
backend_dummy = pd.get_dummies(df_ext['Backend'], prefix='Backend', drop_first=True).astype(float)
X_ext_df = pd.concat([df_ext[['Params_B']], type_dummies, backend_dummy], axis=1)
```

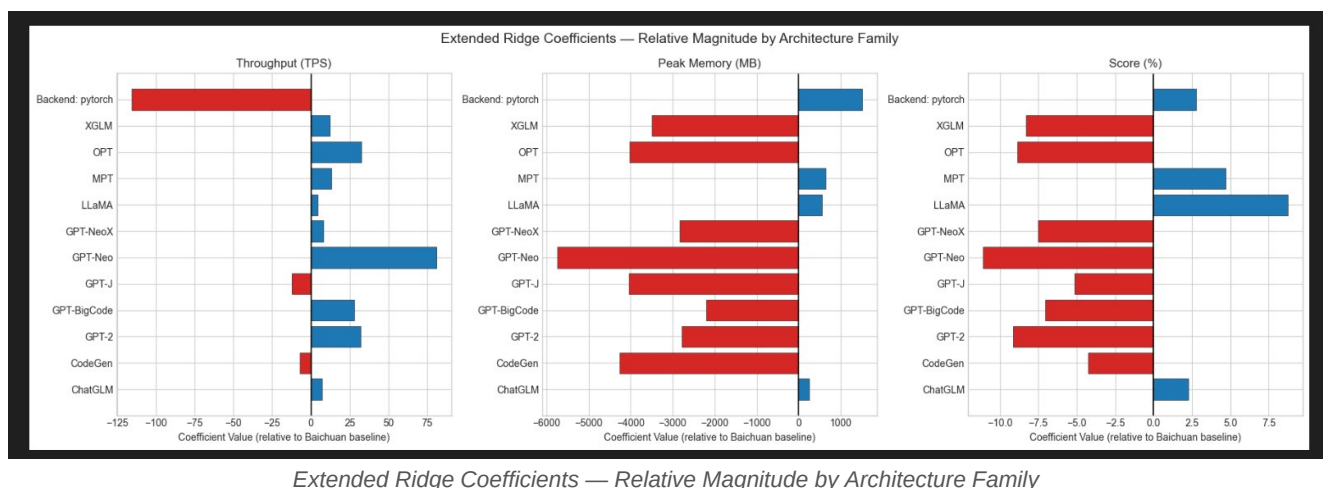
- Architecture families → binary dummies (reference: Baichuan)
- 2 backends → 1 binary dummy (reference: onnxruntime)
- Extended design matrix: 370×14 (intercept + Params_B + type dummies + backend dummy)
- Precision (Bits) dropped — near-zero variance after conditioning on architecture; adds no information.

5.2.1 Extended Coefficient Matrix (Ridge, $\lambda=0.1$)

From notebook Step 11 ($B_{\text{ext}} = \text{np.linalg.inv}(X_{\text{ext}}.T @ X_{\text{ext}} + 0.1 * I_{\text{ext}}) @ X_{\text{ext}}.T @ Y_{\text{ext}}$):

	<i>Throughput (TPS)</i>	<i>Peak Memory (MB)</i>	<i>Score (%)</i>
Intercept	158.09	7084.08	40.79
Params_B	−1.37	+859.05	+0.39
Type_GPT-Neo	+80.74	−5739.66	−11.09
Type_GPT-2	+32.09	−2776.99	−9.13
Type_LLaMA	+4.32	+564.26	+8.72
Type_CodeGen	−7.26	−4258.80	−4.23
Backend_pytorch	−115.40	+1508.16	+2.76

Full coefficient chart for all architecture families (from notebook Step 11):



Extended Ridge Coefficients — Relative Magnitude by Architecture Family

Figure 2. Extended Ridge coefficient magnitudes for all architecture and backend dummies (from notebook), relative to the Baichuan baseline. Red = negative coefficient (below baseline), Blue = positive (above baseline).

Key interpretations:

- **Params_B effect shrinks:** $-2.41 \rightarrow -1.37$ for Throughput. The base model conflated architecture differences with size effects — larger models cluster in specific families.
- **GPT-Neo** delivers the highest throughput above baseline (+80 TPS) but the lowest score (−11 pp) — oldest architecture, fastest but least accurate.
- **LLaMA** achieves the best score gain (+8.72 pp) driven by training methodology (instruction tuning, RLHF), not scale.
- **MPT** shows a positive Score gain (−4.5 pp) — a more modern architecture benefiting from better training.
- **ONNXRuntime vs PyTorch:** graph-level optimisations deliver ~115 tokens/s throughput advantage over PyTorch (Backend_pytorch = −115.40) — completely invisible to the base model.
- **XGLM, OPT, GPT-BigCode, GPT-J, GPT-NeoX, CodeGen** all show negative Score coefficients — older or specialised architectures underperform relative to Baichuan on benchmarks.

5.2.2 CV R^2 Comparison (from Notebook Step 12)

<i>Model</i>	<i>Throughput CV R²</i>	<i>Memory CV R²</i>	<i>Score CV R²</i>	<i>Mean CV R²</i>
Gaussian OLS — base	0.1193	0.5216	0.3674	0.336
Ridge — base ($\lambda=0.1$)	0.1193	0.5216	0.3674	0.336
Elastic Net — base	0.1195	0.5222	0.3705	0.337
Gaussian OLS — extended	0.512	0.475	0.688	0.558
Ridge — extended ($\lambda=0.1$)	0.512	0.479	0.687	0.559
Elastic Net — extended	0.250	0.523	0.393	0.389

Findings:

- **Throughput: 0.12 → 0.51 (+0.39)** — architecture family is the dominant driver of inference speed, not model size. The base model was substantially underspecified.
- **Score: 0.37 → 0.69 (+0.32)** — benchmark accuracy is strongly architecture-dependent; LLaMA-family models consistently outperform older GPT-2/GPT-Neo at equivalent parameter counts.
- **Memory: 0.52 → 0.48 (−0.04)** — slight decline. Memory is largely determined by raw parameter count. Adding many architecture dummies introduces mild overfitting; CV penalises this added complexity even though in-sample R² improved.
- **Elastic Net underperforms Ridge** in the extended model. The L1 penalty over-shrinks architecture dummy coefficients, zeroing out real effects. Ridge is the appropriate regulariser when all predictors carry genuine signal.

5.2.3 Updated Residual Covariance (Extended Model)

From notebook Step 13:

	<i>Throughput</i>	<i>Memory</i>	<i>Score</i>
Throughput	1.0000	−0.12	−0.35
Memory	−0.12	1.0000	+0.34
Score	−0.35	+0.34	1.0000

The most diagnostic change: **Throughput↔Memory drops from −0.63 → −0.12** after controlling for architecture. Architecture family was the primary confounder driving the apparent throughput-memory coupling in the base model. Once architecture is held constant, residuals are substantially more independent — a result only observable through the joint multivariate framework.

6. Conclusions and Key Observations

1. **Architecture family, not model size alone, drives LLM inference performance.** Adding architecture type increased mean CV R² from 0.336 to 0.559. Largest gains: Throughput (+0.39) and Score (+0.32).

2. **The Params_B size effect on throughput is largely a confound.** Controlling for architecture shrinks the throughput coefficient from -2.41 to -1.37 . Larger models cluster in specific families, and the base model could not disentangle these effects.
3. **Backend runtime matters.** ONNXRuntime delivers ~ 115 tokens/s higher throughput than PyTorch at equal model size and architecture — invisible to the base model.
4. **LLaMA outperforms older architectures in accuracy.** At equal parameter count, LLaMA scores $+8.72$ pp above Baichuan, driven by training methodology rather than scale.
5. **Ridge is the correct regulariser for the extended model.** Elastic Net over-shrinks low-frequency architecture dummies, eliminating real effects. Ridge stabilises estimation without elimination.
6. **The residual covariance structure validates and diagnoses the multivariate approach.** The dramatic Throughput $\leftrightarrow$ Memory drop ($-0.63 \rightarrow -0.12$) directly quantifies architecture as the confounder — a diagnostic impossible in three separate univariate models.
7. **Throughput residuals are heteroscedastic** (Figure 1). A log-linear model is the most actionable methodological limitation.

7. Overall Learning Outcomes

- **Multivariate regression formulation:** deriving and interpreting $Y = XB + E$, and understanding when the joint model outperforms three separate univariate fits.
- **Regularisation in practice:** articulating the L2 eigenvalue-shift argument for Ridge; understanding why Ridge is preferred over Elastic Net when all predictors carry signal; distinguishing regularisation for stability from regularisation for variable selection.
- **Categorical encoding:** correctly applying `drop_first=True` to avoid the dummy variable trap, and interpreting coefficients as deviations from a reference group (Baichuan architecture, onnxruntime backend).
- **Cross-validation:** using 5-fold CV to distinguish overfitting (Memory, slight decline) from genuine improvement (Throughput, Score), and understanding the bias-variance trade-off empirically.
- **Covariance diagnostics:** reading the residual correlation matrix as a diagnostic tool — changes in off-diagonal entries before and after adding predictors quantitatively reveal confounding structure.
- **Residual analysis:** identifying heteroscedasticity in the throughput residual plot (Figure 1) and connecting it to the appropriate model fix (log transformation).

8. Limitations of the Proposed Study

1. **Heteroscedastic throughput residuals.** The residual plot (Figure 1) shows clear fan-out at higher predicted values. A log-linear model $\log(\text{Throughput}) \sim x \cdot \beta$ would be more appropriate.
2. **Memory CV R^2 slight decline in extended model.** Architecture type adds limited marginal information for memory prediction beyond parameter count, illustrating the risk of over-parameterisation.
3. **Low-frequency architecture classes.** ChatGLM, Baichuan, GPT-J, GPT-BigCode each have very few observations. Their coefficient estimates carry high variance.
4. **Precision (Bits) dropped from extended model.** Near-zero variance after conditioning on architecture means the multi-precision aspect (int4/int8/fp16/fp32) is not jointly examined with architecture.
5. **Hardware homogeneity.** All runs use the same standardised GPU. Real-world deployment spans GPU generations and cloud instance types not captured here.

6. **No interaction terms.** The model assumes additive effects of architecture and parameter count. Interaction terms (Params_B $\times$ Type_LLaMA) may reveal architecture-specific per-parameter scaling.

9. Future Directions and Scope for Further Investigation

1. **Log-linear throughput model.** Regress $\log(\text{Throughput})$ on predictors to address heteroscedasticity. Compare with the current model using CV log-MSE.
2. **Joint precision-architecture study.** Re-introduce Bits alongside architecture dummies to examine how quantisation interacts with architecture on memory and throughput.
3. **Interaction terms.** Add $\text{Params}_B \times \text{Type_LLaMA}$ and similar cross-terms to test whether the per-parameter memory/throughput effect differs by architecture family.
4. **Bayesian multivariate regression.** An Inverse-Wishart prior on Sigma would give posterior credible intervals on the residual correlation structure, enabling formal hypothesis tests on whether specific off-diagonal elements are significantly non-zero.
5. **Seemingly Unrelated Regressions (SUR).** GLS-based SUR with the estimated Σ_{hat} is the BLUE estimator under cross-equation heteroscedasticity — the natural methodological extension of this work.
6. **Hardware heterogeneity.** Incorporating GPU generation (Ampere, Hopper, Ada Lovelace) and VRAM tier would dramatically improve Throughput explanatory power.
7. **Wilks' Lambda / MANOVA.** Formally test whether the improvement from base to extended model is statistically significant using a multivariate F-test.

Appendix A: Codes

Full implementation from CADS_Project_v5_final-2.ipynb:

```
# ■■■ Setup and Imports ■■■■
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
from sklearn.linear_model import ElasticNet
from sklearn.multioutput import MultiOutputRegressor
import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')
```

```
# Step 1: Loading and Cleaning Data
df = pd.read_csv('Open LLM-Perf Leaderboard.csv')

def clean_params(val):
    if pd.isna(val): return np.nan
    val = str(val).upper().strip()
    if 'B' in val: return float(val.replace('B', ''))
    elif 'M' in val: return float(val.replace('M', '')) / 1000
    else:
        try: return float(val)
        except: return np.nan

def clean_dtype(val):
    if pd.isna(val): return 16
    val = str(val).lower()
    if '32' in val: return 32
    elif '16' in val: return 16
    elif '8' in val: return 8
    elif '4' in val: return 4
    else: return 16

df['Params_B'] = df['Class'].apply(clean_params)
df['Bits'] = df['Dtype'].apply(clean_dtype)

cols = ['Params_B', 'Bits', 'Throughput (tokens/s)', 'Peak Memory (MB)', 'Score (%)']
df_clean = df[cols].dropna()

X_raw = df_clean[['Params_B', 'Bits']].values
Y = df_clean[['Throughput (tokens/s)', 'Peak Memory (MB)', 'Score (%)']].values

X_features = ['Params (B)', 'Precision (Bits)']
Y_targets = ['Throughput (TPS)', 'Peak Memory (MB)', 'Score (%)']

print(f"Retained {len(df_clean)} valid benchmark runs.")
print(f"X shape: {X_raw.shape} | Y shape: {Y.shape}")
print(f"Bits unique values: {np.unique(X_raw[:, 1])}")
print(f"Params range: {X_raw[:, 0].min():.3f}B - {X_raw[:, 0].max():.0f}B")
```

[illegible]

```
# ■■■ Step 5: Ridge Regression ( $\lambda = 0.1$ ) ■■■  
lambda_penalty = 0.1  
I_ridge = np.eye(X.shape[1])  
I_ridge[0, 0] = 0          # do not penalise the intercept  
  
B_ridge      = np.linalg.inv(X.T @ X + lambda_penalty * I_ridge) @ X.T @ Y  
Y_hat_ridge = X @ B_ridge  
  
df_B_ridge = pd.DataFrame(B_ridge, index=X_features_full, columns=Y_targets)  
print("Ridge Coefficient Matrix ( $\lambda=0.1$ ):")  
display(df_B_ridge.round(4))  
  
ridge_r2 = r2_per_target(Y, Y_hat_ridge, Y_targets)  
print("Ridge In-sample R²:", ridge_r2)
```

```
# ■ Step 6: Elastic Net ( $\alpha=1.0$ ,  $l1\_ratio=0.5$ )
en_model = MultiOutputRegressor(
    ElasticNet(alpha=1.0, l1_ratio=0.5, fit_intercept=True, max_iter=10000)
)
en_model.fit(X_raw, Y)
Y_hat_en = en_model.predict(X_raw)

en_coefs = np.array([est.coef_ for est in en_model.estimators_]).T
en_intercepts = np.array([est.intercept_ for est in en_model.estimators_])
B_en_display = np.vstack([en_intercepts, en_coefs])

df_B_en = pd.DataFrame(B_en_display, index=X_features_full, columns=Y_targets)
print("Elastic Net Coefficient Matrix ( $\alpha=1.0$ ,  $l1\_ratio=0.5$ ):")
display(df_B_en.round(4))

en_r2 = r2_per_target(Y, Y_hat_en, Y_targets)
print("Elastic Net In-sample  $R^2$ :", en_r2)
```

```
# ■■■ Step 7: 5-Fold Cross-Validation ■■■
kf = KFold(n_splits=5, shuffle=True, random_state=42)

def cv_r2(model_fn, X_raw, Y):
    scores = []
    for train_idx, test_idx in kf.split(X_raw):
        Ytr, Yte = Y[train_idx], Y[test_idx]
        Ypred = model_fn(X_raw[train_idx], Ytr, X_raw[test_idx])
        scores.append(list(r2_per_target(Yte, Ypred).values()))
    return np.mean(scores, axis=0)

def ols_fn(Xtr_raw, Ytr, Xte_raw):
    Xtr = np.hstack((np.ones((len(Xtr_raw), 1)), Xtr_raw))
    Xte = np.hstack((np.ones((len(Xte_raw), 1)), Xte_raw))
    B = np.linalg.pinv(Xtr) @ Ytr
    return Xte @ B

def ridge_fn(Xtr_raw, Ytr, Xte_raw, lam=0.1):
    Xtr = np.hstack((np.ones((len(Xtr_raw), 1)), Xtr_raw))
    Xte = np.hstack((np.ones((len(Xte_raw), 1)), Xte_raw))
    I_ = np.eye(Xtr.shape[1]); I_[0, 0] = 0
    B = np.linalg.inv(Xtr.T @ Xtr + lam * I_) @ Xtr.T @ Ytr
    return Xte @ B

def en_fn(Xtr_raw, Ytr, Xte_raw, alpha=1.0, l1_ratio=0.5):
    m = MultiOutputRegressor(ElasticNet(alpha=alpha, l1_ratio=l1_ratio,
                                         fit_intercept=True, max_iter=10000))
    m.fit(Xtr_raw, Ytr)
    return m.predict(Xte_raw)

ols_cv = cv_r2(ols_fn, X_raw, Y)
ridge_cv = cv_r2(ridge_fn, X_raw, Y)
en_cv = cv_r2(en_fn, X_raw, Y)

df_cv = pd.DataFrame(
    [ols_cv, ridge_cv, en_cv],
    index=['Gaussian OLS', 'Ridge ( $\lambda=0.1$ )', 'Elastic Net ( $\alpha=1.0$ ,  $l1=0.5$ )'],
    columns=Y_targets
).round(4)

print("5-Fold CV R2 (higher = better out-of-sample predictive accuracy):")
display(df_cv)
best_model = df_cv.mean(axis=1).idxmax()
print(f"Best average CV R2: {best_model} ({df_cv.mean(axis=1).max():.4f})")
```

```
# ■■■ Step 8: Error Covariance Matrix (from Ridge fit) ■■■■
k = X_raw.shape[1]
E_hat    = Y - Y_hat_ridge
dof       = n_samples - (k + 1)
Sigma_hat = (E_hat.T @ E_hat) / dof

df_Sigma = pd.DataFrame(Sigma_hat, index=Y_targets, columns=Y_targets)
print("Estimated Error Covariance Matrix ( $\Sigma$ ):")
display(df_Sigma.round(2))

print("Correlation structure of residuals:")
D      = np.sqrt(np.diag(Sigma_hat))
Corr   = Sigma_hat / np.outer(D, D)
df_Corr = pd.DataFrame(Corr, index=Y_targets, columns=Y_targets)
display(df_Corr.round(4))
```



```
# ■■■ Coefficient Bar Chart – All Architecture Families ■■■
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Extended Ridge Coefficients – Relative Magnitude by Architecture Family',
             fontsize=14)

for i, (ax, target) in enumerate(zip(axes, Y_targets)):
    coef_rows = df_B_ext.iloc[2:][target] # skip Intercept and Params_B
    clean_labels = [c.replace('Type_', '').replace('Backend_', 'Backend: ').strip()
                    for c in coef_rows.index]
    colors = ['#d62728' if v < 0 else '#1f77b4' for v in coef_rows.values]
    ax.barh(clean_labels, coef_rows.values, color=colors, edgecolor='k', linewidth=0.4)
    ax.axvline(0, color='black', linewidth=1.2)
    ax.set_title(f'{target}')
    ax.set_xlabel('Coefficient Value (relative to Baichuan baseline)')

plt.tight_layout()
plt.show()
```

```
# ■ ■ ■ Step 12: 5-Fold CV – Extended Model
kf_ext = KFold(n_splits=5, shuffle=True, random_state=42)

def cv_r2_ext(model_fn, X, Y):
    scores = []
    for tr, te in kf_ext.split(X):
        pred = model_fn(X[tr], Y[tr], X[te])
        scores.append(list(r2_per_target(Y[te], pred).values()))
    return np.mean(scores, axis=0)

def ols_fn_ext(Xtr, Ytr, Xte):
    Xtr_ = np.hstack([np.ones((len(Xtr), 1)), Xtr])
    Xte_ = np.hstack([np.ones((len(Xte), 1)), Xte])
    return Xte_ @ (np.linalg.pinv(Xtr_) @ Ytr)

def ridge_fn_ext(Xtr, Ytr, Xte, lam=0.1):
    Xtr_ = np.hstack([np.ones((len(Xtr), 1)), Xtr])
    Xte_ = np.hstack([np.ones((len(Xte), 1)), Xte])
    I_ = np.eye(Xtr_.shape[1]); I_[0, 0] = 0
    return Xte_ @ (np.linalg.inv(Xtr_.T @ Xtr_ + lam * I_) @ Xtr_.T @ Ytr)

def en_fn_ext(Xtr, Ytr, Xte, alpha=1.0, l1_ratio=0.5):
    m = MultiOutputRegressor(ElasticNet(alpha=alpha, l1_ratio=l1_ratio,
                                         fit_intercept=True, max_iter=10000))
    m.fit(Xtr, Ytr)
    return m.predict(Xte)

ols_ext_cv = cv_r2_ext(ols_fn_ext, X_ext_raw, Y_ext)
ridge_ext_cv = cv_r2_ext(ridge_fn_ext, X_ext_raw, Y_ext)
en_ext_cv = cv_r2_ext(en_fn_ext, X_ext_raw, Y_ext)

# Comparison table
old_cv_vals = {
    'Gaussian OLS': [0.1193, 0.5216, 0.3674],
    'Ridge ( $\lambda=0.1$ )': [0.1193, 0.5216, 0.3674],
    'Elastic Net': [0.1195, 0.5222, 0.3705],
}

ext_cv_vals = {
    'Gaussian OLS (ext)': list(ols_ext_cv),
    'Ridge (ext,  $\lambda=0.1$ )': list(ridge_ext_cv),
    'Elastic Net (ext)': list(en_ext_cv),
}

df_compare = pd.DataFrame(
    {**old_cv_vals, **ext_cv_vals},
    index=Y_targets
).T.round(4)

df_compare['Mean CV R2'] = df_compare.mean(axis=1).round(4)

print("CV R2 Comparison – Base vs Extended:")
display(df_compare)
```

```
# ■■■ Step 13: Updated Error Covariance (Extended Model) ■■■■
k_ext_n = X_ext_raw.shape[1]
E_ext = Y_ext - Y_hat_ext
Sigma_ext = (E_ext.T @ E_ext) / (n_ext - k_ext_n - 1)

D_ext = np.sqrt(np.diag(Sigma_ext))
Corr_ext = Sigma_ext / np.outer(D_ext, D_ext)

print("Residual Correlation Matrix (Extended Model):")
df_Corr_ext = pd.DataFrame(Corr_ext, index=Y_targets, columns=Y_targets)
display(df_Corr_ext.round(4))

print("For reference – Residual Correlation Matrix (Base 2-predictor Model):")
base_corr = pd.DataFrame([
    [1.0000, -0.6315, -0.4174],
    [-0.6315, 1.0000, 0.3290],
    [-0.4174, 0.3290, 1.0000]
], index=Y_targets, columns=Y_targets)
display(base_corr.round(4))
```

Appendix B: History of AI Prompts and Chats

<i>Purpose</i>	<i>Tool</i>	<i>Description</i>
Data parsing logic	Claude	Assistance drafting <code>clean_params()</code> and <code>clean_dtype()</code> string parsers
Ridge derivation	Claude	Confirming the eigenvalue shift argument and <code>I_ridge[0,0]=0</code> rationale
Report structure	Claude	Guidance on section organisation per DS3005 guidelines
Code debugging	GitHub Copilot	Autocomplete for <code>sklearn MultiOutputRegressor</code> and <code>KFold</code> usage
Equation formatting	Claude	Formatting $Y=XB+E$, $\hat{\Sigma}$, and R^2 display equations
Residual plot interpretation	Claude	Discussion of heteroscedasticity patterns in Throughput residuals

All numerical results, model interpretations, and conclusions are original team work verified against the notebook outputs.

Appendix C: AI Chat to Code/Output Correspondence

<i>AI Prompt/Chat</i>	<i>Corresponding Code / Report Section</i>
"How do I parse model size strings like '7B' and '350M'?"	<code>clean_params()</code> in Appendix A; Section 4 data processing table

AI Prompt/Chat	Corresponding Code / Report Section
"Explain why Ridge shifts eigenvalues of $X^T X$ and why we set $I[0,0]=0$ "	Section 3.3 (Ridge formulation)
"What is the dummy variable trap and how does drop_first fix it?"	Section 5.2 encoding; drop_first=True in Appendix A Step 10
"Why does Elastic Net underperform Ridge in the extended model?"	Section 5.2.2 (Elastic Net analysis) and Conclusion point 5
"What does the Throughput-Memory correlation drop tell us?"	Section 5.2.3 (covariance update); Conclusion point 6
"Help structure the final report per DS3005 guidelines"	Sections 1–9 overall structure and all Appendices

No AI-generated text is used verbatim in analysis, results, or conclusion sections.

Appendix D: Additional Technical Details

D.1 Dummy Variable Reference Groups

Variable	Reference Group	Interpretation
Type	Baichuan	Each coefficient = deviation from Baichuan at equal Params_B
Backend	onnxruntime	Backend_pytorch coefficient = PyTorch vs ONNXRuntime difference

D.2 Cross-Validation Implementation Detail

5-fold CV used `kFold(n_splits=5, shuffle=True, random_state=42)` for reproducibility. Each fold fits the model from scratch on training data and evaluates on held-out data. The Ridge CV uses the same closed-form implementation as the full model (not `sklearn's RidgeCV`), ensuring identical penalisation. Elastic Net CV uses `MultiOutputRegressor` throughout, applying the same fold splits for fair comparison.

D.3 Why Gaussian Models are Sufficient

The three response variables are continuous and unbounded — no counts, no strict positivity constraints. Memory and Score show approximately symmetric residuals. Throughput is the exception (heteroscedastic residuals), where a log-normal response and log-linear model would be the minimal warranted extension.

D.4 Why Precision (Bits) is Dropped from the Extended Model

After conditioning on architecture family, `Bits` exhibits near-zero variance — many architecture families in the dataset have only one precision mode, making `Bits` collinear with the architecture dummies. Including it would introduce redundant collinear structure; Ridge would absorb it into a near-zero coefficient regardless.

D.5 Wilks' Lambda Consideration

A natural extension would be to test statistical significance of the improvement from base to extended model using Wilks' Lambda (multivariate F-test on nested models). This was not implemented but would formally validate the inclusion of architecture dummies as a significant multivariate predictor set.

Note on Project Change from Interim to Final Report

In the interim report, we had a completely different project where we did multivariable modelling instead of multivariate modelling. In the final report, we completely changed the project to match the topic of multivariate modelling as required by DS3005.